VAPT 4

Advanced Exploitation Lab Report

Name: Vijayashree B
Domain: Cybersecurity (VAPT)
Date of Submission: 03.04.26

1. Objective

The objective of this lab is to simulate real-world attack scenarios by chaining exploits, developing custom Proof-of-Concept (PoC) scripts, and bypassing security defenses such as ASLR using advanced exploitation techniques.

2. Tools Used

- Metasploit Framework
- Python
- Ghidra

3. Lab Environment

- Target Machine: VulnHub VM
- Target IP: 192.168.56.108
- Attacker Machine: Kali Linux

4. Exploit Log

Exploit ID	Description	Target IP	Status	Payload
007	XSS to RCE Chain	192.168.56.108	Success	Meterpreter

5. Steps Performed

- Scanned the target using Metasploit auxiliary modules.
- Identified a vulnerable WordPress plugin.
- Used `exploit/multi/http/wordpress_plugin_rce`.
- Configured payload as Meterpreter reverse shell.
- Executed exploit and gained shell access.
- Maintained persistence and verified access.

6. Binary Analysis using Ghidra

The target binary (vuln) was analyzed using Ghidra. The decompiled code revealed the use of an unsafe function:

```
char buffer[112];
gets(buffer);
puts(buffer);
```

Findings

- The gets() function does not perform bounds checking.
- This leads to a buffer overflow vulnerability.
- Attackers can overwrite the return address and gain control of execution flow.

7. Custom Proof-of-Concept (PoC)

A Python-based exploit was developed by modifying an existing Exploit-DB PoC to target the buffer overflow vulnerability.

PoC Summary (50 words)

A Python exploit was modified to trigger a buffer overflow by sending crafted input exceeding the buffer size. The payload overwrites the return address and redirects execution flow. The script automates exploitation, allowing reliable triggering of the vulnerability and demonstrating control over the target binary execution.

8. Bypass Technique (ROP for ASLR)

Bypass Summary (50 words)

Return-Oriented Programming (ROP) was used to bypass Address Space Layout Randomization (ASLR). Gadgets were identified within the binary and linked libraries to construct a ROP chain. This allowed controlled execution without injecting shellcode, effectively bypassing memory protection mechanisms and gaining execution control over the vulnerable program.

9. Remediation

- Update all WordPress plugins regularly
- Avoid unsafe functions like `gets()` in code
- Enable Web Application Firewall (WAF)
- Implement ASLR and stack canaries
- Conduct regular vulnerability assessments

10. Conclusion

The lab successfully demonstrated exploitation techniques including exploit chaining, custom PoC development, and bypassing modern defenses. The findings highlight the importance of secure coding practices and timely patching to prevent critical vulnerabilities.